Assignment Machine Learning 2

Name: Justin Christian Kenan

Student ID: 2802399463

Class: LC01

Multiple Linear Regression

Libraries

In [12]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Dataset

In [13]:
```
dataset = pd.read_csv('Housingg.csv')
dataset.head(10)
```

Out[13]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hot
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	
5	10850000	7500	3	3	1	yes	no	yes	
6	10150000	8580	4	3	4	yes	no	no	
7	10150000	16200	5	3	2	yes	no	no	
8	9870000	8100	4	1	2	yes	yes	yes	
9	9800000	5750	3	2	4	yes	yes	no	

Cleaning null data

In [14]:
```
clean_data = dataset.dropna(subset=['area', 'bedrooms', 'price'])
```

Define feature (x) and target (y)

In [15]:
```
x = clean_data[['area', 'bedrooms']]
y = clean_data['price']
```

Normalize

In [16]:
```
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
```

Spliting data into 80:20 (Training : Test)

In [17]:
```
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.2, ran
```

Fitting model MLR

In [18]:
```
regressor = LinearRegression()
regressor.fit(x_train, y_train)
```

Out[18]:

▾ LinearRegression

▸ Parameters

Prediction training set and test set

In [19]:
```
y_pred_train = regressor.predict(x_train)
y_pred_test = regressor.predict(x_test)
```

Evaluation metrics

In [20]:
```
def metrics(y_true, y_pred, label):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f'{label} Metrics:')
    print(f'RMSE: {rmse:.2f}')
    print(f'MAE: {mae:.2f}')
    print(f'R2: {r2:.3f}\n')

metrics(y_train, y_pred_train, 'Training Set')
metrics(y_test, y_pred_test, 'Test Set')
```

```
Training Set Metrics:
RMSE: 1393261.87
MAE: 1015508.63
R2: 0.370

Test Set Metrics:
RMSE: 1811125.78
MAE: 1381158.90
R2: 0.351
```

Based on GSLC 9-10

How to avoid the underfitting in Model Supervise Learning ?

1. Decrease Regularization Regularization method such as L1 regularization and Lasso regularization are used to reduce the variance with model by applying a penalty to the input parameters with the larger coefficient that will help to reduce noise and outliers with a model. If the penalty parameter is too large, the model will too rigid and causes underfitting, with reducing this parameter will give the model more space to learn with detail.

2. Feature Selection Specific features are used to determine a given outcome, but if predictive feature are not enough, the feature with more important must be added. By adding new and important features from the dataset, the model has more variables to understand the relationship between input and target.

3. Increase the duration of training time Stopping the training process too soon in algorithms that use iteration will prevent the model from finding the optimal pattern. With increasing the number of epochs can support the model to reach a better convergence point.

Explain two types of regularization techniques !

1. Lasso Regression LASSO (Least Absolute Shrinkage and Selection Operator) regression is a regression model which uses the L1 Regularization technique, this regression model adds the absolute value of magnitude of the coefficient as a penalty to loss function that will shrink some coefficients to zero which helps in selecting the most important features.

2. Ridge Regression Ridge Regression is a regression model that uses the L2 Regularization technique with adds the squared magnitude of the coefficient as a penalty to loss function. By shrinking the coefficient of correlated features instead of eliminate make this regression model can handle multicollinearity.

Calculate output k-fold Cross-Validation for dataset = [[10], [9], [8], [7], [6], [5], [4], [3], [2], [1]]

If use k=5, each fold will be 2 data, because total data / k = 10 / 5 = 2 data

Output k-fold

1. Iteration 1: Test: [10, 9] | Train: [8,7,6,5,4,3,2,1]
2. Iteration 2: Test: [8, 7] | Train: [10,9,6,5,4,3,2,1]
3. Iteration 3: Test: [6, 5] | Train: [10,9,8,7,4,3,2,1]
4. Iteration 4: Test: [4, 3] | Train: [10,9,8,7,6,5,2,1]
5. Iteration 5: Test: [2, 1] | Train: [10,9,8,7,6,5,4,3]

In [21]:
```
from sklearn.model_selection import KFold
import numpy as np

dataset = np.array([[10], [9], [8], [7], [6], [5], [4], [3], [2], [1]])

kValue = KFold(n_splits=5)

for i, (train_index, test_index) in enumerate(kValue.split(dataset)):
    print(f"Iteration {i+1}:")
    print(f"   Train: {dataset[train_index].flatten()}")
    print(f"   Test : {dataset[test_index].flatten()}")
```

```
Iteration 1:
   Train: [8 7 6 5 4 3 2 1]
   Test : [10  9]
Iteration 2:
   Train: [10  9  6  5  4  3  2  1]
   Test : [8 7]
Iteration 3:
   Train: [10  9  8  7  4  3  2  1]
   Test : [6 5]
Iteration 4:
   Train: [10  9  8  7  6  5  2  1]
   Test : [4 3]
Iteration 5:
   Train: [10  9  8  7  6  5  4  3]
   Test : [2 1]
```